

Resolve Merge Conflicts

Created a new files in both branches with same names with different content and created a new merge conflict

Then edited the merge conflict file and then git add .

Git commit

Then merge conflict has been resolved

```
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both added:      firsttext.txt

no changes added to commit (use "git add" and/or "git commit -a")

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave|MERGING)
$ vi firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave|MERGING)
$ git add .

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave|MERGING)
$ git commit -m "merge conflict file edited and committed"
[slave 87fcc0] merge conflict file edited and committed

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git merge main
Already up to date.

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ ls
README.md  firsttext.txt
```

```
MINGW64:/c/Users/Rajes/Desktop/diamond
<<<<<< HEAD
new text file in slave brnch for merge conflicts
=====
new text created in main branch

>>>>>> main
~
```

Deleting a branch and recovering using reflog

Use git branch -d branch name to delete the branch

Then do git reflog to see the last is of branch change

Then Git checkout -b branchname BRANCHID

```
rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git branch slave2

rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git checkout slave2
switched to branch 'slave2'

rajes@charanreddy MINGW64 ~/Desktop/diamond (slave2)
$ ls
README.md  firsttext.txt

rajes@charanreddy MINGW64 ~/Desktop/diamond (slave2)
$ cat firsttext.txt
new text created in main branch

rajes@charanreddy MINGW64 ~/Desktop/diamond (slave2)
$ git branch -d slave2
error: Cannot delete branch 'slave2' checked out at 'c:/Users/Rajes/Desktop/diamond'

rajes@charanreddy MINGW64 ~/Desktop/diamond (slave2)
$ git checkout main
switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git branch -d slave2
deleted branch slave2 (was f0996a5).

rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git branch
* main
  slave
```

```

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git branch --list
  f0996a5
  list
* main
  slave

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git checkout slave2
error: pathspec 'slave2' did not match any file(s) known to git

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git checkout -b slave2 f0996a5
warning: refname 'f0996a5' is ambiguous.
warning: refname 'f0996a5' is ambiguous.
Switched to a new branch 'slave2'

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave2)
$

```

Undo Wrong Push

- Push a wrong commit to GitHub, then undo it without losing history

This branch is 5 commits ahead of `main`.

 **saicharanreddy5678** Revert "commit for the wrong push undo" 

 README.md	Initial commit
 firsttext.txt	merge conflict file edited and committed

 **slave** 2 Branches 0 Tags

This branch is 4 commits ahead of `main`.

 **saicharanreddy5678** commit for the wrong push undo

 README.md	Initial commit
 firsttext.txt	merge conflict file edited and committed
 newslavefile	commit for the wrong push undo

I have pushed a wrong commit and i used “git revert commitid “of the wrong commit

And used git "push" to make changes in GitHub

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log
commit ee1f3320a661cd742428b46530da2fe0a7c155dd (HEAD -> slave, origin/
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 11:57:08 2026 +0530

    Revert "commit for the wrong push undo"

    This reverts commit d8ff6e02da8207351f87eb252da9ef73113a8cf3.
```

```
commit 99727bce1ef01ee70970770ba0212c919198c (origin/main, origin/HEAD)
Author: saicharanreddy5678 <saicharanreddy5678@outlook.com>
Date:   Wed May 20 11:21:02 2026 +0530

    Initial commit

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ vi newslavefile

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git status
On branch slave
Your branch is up to date with 'origin/slave'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    newslavefile

nothing added to commit but untracked files present (use "git add" to track)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git add .
warning: in the working copy of 'newslavefile', LF will be replaced by CRLF the

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git commit -m "commit for the wrong push undo"
[slave d8ff6e0] commit for the wrong push undo
1 file changed, 1 insertion(+)
create mode 100644 newslavefile

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 385 bytes | 385.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
```

Amend a Commit

First committed a file and then i have added a new file to the last commit

Then used

Git commit –ammend to attach new file commit to the last commit

With out creating a new commit

```
$ touch file2.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ vi file2.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ touch 2.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git .gitignore
git: '.gitignore' is not a git command. See 'git --help'.

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ vi .gitignore

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git status
On branch slave
Your branch is up to date with 'origin/slave'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        2.txt

nothing added to commit but untracked files present (use "git add" to track)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git commit -m "creation of ammdend first "
[slave 1789fbf] creation of ammdend first
2 files changed, 2 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 2.txt
```

```

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ touch 45.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ ls
2.txt 45.txt README.md file2.txt firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git status
On branch slave
Your branch is ahead of 'origin/slave' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    45.txt

nothing added to commit but untracked files present (use "git add" to track)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log
commit 1789fbfb9e8a96d00a014f9c9749e5f318649850 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 12:16:35 2026 +0530

    creation of ammend first

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log -4
commit 0f41937b7d23a2365da0e416cbdb32d591652a6 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 12:16:35 2026 +0530

    creation of ammend first and 2nd commit of thius same commit for
    45.txt

```

Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.

I have a file in my subbranch and i want to copy this only 1 file to my main branch

So, i will be using **“git cherry-pick commitid”**

For this i need to have the file commit id with commit id of the file i can cherry-pick the file from one branch to another

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log -3
commit d0ad53d1f344f7006f9ca9f0fbe01ce38ef528de (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 12:28:36 2026 +0530

    commit for file cherryfile

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git cherry-pick d0ad53d
[main cc0b465] commit for file cherryfile
Date: Wed May 20 12:28:36 2026 +0530
1 file changed, 1 insertion(+)
create mode 100644 cherryfile

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ ls
README.md  cherryfile  file2.txt  firsttext.txt
```

Interactive Rebase

- Reorder and squash multiple commits into a single clean commit

Here i have multiple commits in my branch so i want to combine multiple commits to a single commit

Git rebase -i head~n (n =number of commits)

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log
commit ec5029d5577562829b66bee01641eecfecc2975a (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 12:45:10 2026 +0530

    commit

    commit for the wrong push undo

    Revert "commit for the wrong push undo"

    This reverts commit d8ff6e02da8207351f87eb252da9ef73113a8cf3.

    creation of ammend first and 2nd commit of thius same commit for
    45.txt

    commit for file cherryfile
```

Tagging & Release

I have created a new tag v1.1.1 using **git tag v1.1.1**

Then deleted using **git tag -d v1.1.1**

When deleted it show its id using the id i can restore the tag

“Git tag v1.1.1 hashid” using this i restored tag

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag v1.1.1

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag
v1.1.1

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag -d v1.1.1
Deleted tag 'v1.1.1' (was c87d94c)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git reflog
c87d94c (HEAD -> slave) HEAD@{0}: commit (amend): commit

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag v1.1.1 c87d94c

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git tag
v1.1.1

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$
```

Clone with Sparse Checkout

- Clone only a subdirectory of a repo using sparse checkout

Reset vs Revert Challenge

- Demonstrate the difference between `git reset --hard` and `git revert` in a repo

```

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git status
On branch slave
Your branch and 'origin/slave' have diverged,
and have 1 and 4 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   45.txt

no changes added to commit (use "git add" and/or "git commit -a")

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git add .

```

```

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git status
On branch slave
Your branch and 'origin/slave' have diverged,
and have 3 and 4 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)

nothing to commit, working tree clean

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ ls
2.txt  45.txt  README.md  cherryfile  file2.txt  firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ cat 45.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$ git log -2
commit a0bd5843d2db518b304e11b2e326fee8ad7950da (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 14:22:12 2026 +0530

    Revert "commit for revert exp"

    This reverts commit f617e854ba865fffd43858d80b36236c625a1927.

commit f617e854ba865fffd43858d80b36236c625a1927
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 14:21:46 2026 +0530

    commit for revert exp

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
$

```

Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it

If i want to make changes in particular and i want to head into the commit and to detach head i have used **git checkout commitid**

Then to work on other file without changing this i will be adding this to new branch using **git branch -b branchname commitid**

And when it is detached whatever changes or commits i have done in detached head it wont save if i switch to old branch if i add this commit to new branch then this commits will be saved

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
```

```
$ git checkout 36c6cb6ed
```

```
Note: switching to '36c6cb6ed'.
```

```
You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.
```

```
If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:
```

```
git switch -c <new-branch-name>
```

```
Or undo this operation with:
```

```
git switch -
```

```
Turn off this advice by setting config variable advice.detachedHead to false
```

```
HEAD is now at 36c6cb6 using for detached head using commit id
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond ((36c6cb6...))
```

```
$
```

```
$ cat 45.txt  
new text file nd jdncjbdc
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond ((1d58652...))
```

```
$ git branch main  
fatal: a branch named 'main' already exists
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond ((1d58652...))
```

```
$ git switch slave  
warning: you are leaving 1 commit behind, not connected to  
any of your branches:
```

```
1d58652 mm
```

If you want to keep it by creating a new branch, this may be a good time to do so with:

```
git branch <new-branch-name> 1d58652
```

```
Switched to branch 'slave'  
Your branch and 'origin/slave' have diverged,  
and have 4 and 4 different commits each, respectively.  
(use "git pull" to merge the remote branch into yours)
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
```

```
$ ls  
2.txt 45.txt README.md cherryfile file2.txt firsttext.txt
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
```

```
$ cat 45.txt  
new text file
```

```
Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave)
```

```
$
```

```
MINGW64/c/Users/Rajes/Desktop/diamond
Rajes@charanreddy MINGW64 ~/Desktop/diamond ((36c6cb6...))
$ git branch slave2

Rajes@charanreddy MINGW64 ~/Desktop/diamond ((36c6cb6...))
$ git branch slave2 36c6cb
fatal: a branch named 'slave2' already exists

Rajes@charanreddy MINGW64 ~/Desktop/diamond ((36c6cb6...))
$ git branch slave3 36c6cb

Rajes@charanreddy MINGW64 ~/Desktop/diamond ((36c6cb6...))
$ git switch slave3
Switched to branch 'slave3'
M   45.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave3)
$ git branch -d slave2
Deleted branch slave2 (was 36c6cb6).

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave3)
$ git branch
  f0996a5
* list
  main
  slave
* slave3

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave3)
$ ls
2.txt  45.txt  README.md  cherryfile  file2.txt  firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave3)
$ cat 45.txt
new text file and or is the

Rajes@charanreddy MINGW64 ~/Desktop/diamond (slave3)
$
```

Git Hooks Challenge

- Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).
- I have created a new file named pre-commit in .git/hooks
- And in pre-commit i have my content that any commits done should have commit message JIRA-XXX

And i have changed pre-msg content to simply commit with JIRA-XXX as first letter for commit message

```
Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git commit -m "committed after creating pre-msg"
X ERROR: Invalid commit message format.
Message must start with 'JIRA-XXX' (e.g., JIRA-123 Fix login bug)

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git commit -m "JIRA-12 committed after creating pre-msg"
[main 1e57c6a] JIRA-12 committed after creating pre-msg
1 file changed, 2 insertions(+)
create mode 100644 a
```

```

MINGW64/c/Users/Rajes/Desktop/diamond
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:saicharanreddy5678/diamond.git
35724be..cc0b465  main -> main

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ ls -a
./ ../ .git/ README.md cherryfile firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ cd .git

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git (GIT_DIR!)
$ ls
COMMIT_EDITMSG  ORIG_HEAD  description  hooks/  info/  objects/  refs/
HEAD           config     fsmonitor--daemon/  index   logs/  packed-refs

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git (GIT_DIR!)
$ cd hooks

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*  pre-applypatch.sample*  pre-rebase.sample*  update.sample*
commit-msg.sample*     pre-commit.sample*      pre-receive.sample*
fsmonitor-watchman.sample*  pre-merge-commit.sample*  prepare-commit-msg.sample*
post-update.sample*      pre-push.sample*         push-to-checkout.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ touch pre-commit

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*  pre-applypatch.sample*  pre-push.sample*  push-to-checkout.sample*
commit-msg.sample*     pre-commit              pre-rebase.sample*  update.sample*
fsmonitor-watchman.sample*  pre-commit.sample*      pre-receive.sample*
post-update.sample*      pre-merge-commit.sample*  prepare-commit-msg.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ vi
Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
git config hooks.allownonascii true
EOF
    exit 1
fi

# If there are whitespace errors, print the offending file names and fail.
exec git diff-index --check --cached $against --

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*  pre-applypatch.sample*  pre-push.sample*  push-to-checkout.sample*
commit-msg.sample*     pre-commit              pre-rebase.sample*  update.sample*
fsmonitor-watchman.sample*  pre-commit.sample*      pre-receive.sample*
post-update.sample*      pre-merge-commit.sample*  prepare-commit-msg.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ vi pre-commit

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ ll
total 46
-rwxr-xr-x 1 Rajes 197609 478 May 20 11:27 applypatch-msg.sample*
-rwxr-xr-x 1 Rajes 197609 896 May 20 11:27 commit-msg.sample*
-rwxr-xr-x 1 Rajes 197609 4726 May 20 11:27 fsmonitor-watchman.sample*
-rwxr-xr-x 1 Rajes 197609 189 May 20 11:27 post-update.sample*
-rwxr-xr-x 1 Rajes 197609 424 May 20 11:27 pre-applypatch.sample*
-rwxr-xr-x 1 Rajes 197609 399 May 20 15:01 pre-commit*
-rwxr-xr-x 1 Rajes 197609 1643 May 20 11:27 pre-commit.sample*
-rwxr-xr-x 1 Rajes 197609 416 May 20 11:27 pre-merge-commit.sample*
-rwxr-xr-x 1 Rajes 197609 1374 May 20 11:27 pre-push.sample*
-rwxr-xr-x 1 Rajes 197609 4898 May 20 11:27 pre-rebase.sample*
-rwxr-xr-x 1 Rajes 197609 544 May 20 11:27 pre-receive.sample*
-rwxr-xr-x 1 Rajes 197609 1492 May 20 11:27 prepare-commit-msg.sample*
-rwxr-xr-x 1 Rajes 197609 2783 May 20 11:27 push-to-checkout.sample*
-rwxr-xr-x 1 Rajes 197609 3650 May 20 11:27 update.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ chmod +x pre-commit

```

```
MINGW64/c/Users/Rajes/Desktop/diamond
-rwxr-xr-x 1 Rajes 197609 189 May 20 11:27 post-update.sample*
-rwxr-xr-x 1 Rajes 197609 424 May 20 11:27 pre-apppatch.sample*
-rwxr-xr-x 1 Rajes 197609 399 May 20 15:01 pre-commit*
-rwxr-xr-x 1 Rajes 197609 1643 May 20 11:27 pre-commit.sample*
-rwxr-xr-x 1 Rajes 197609 416 May 20 11:27 pre-merge-commit.sample*
-rwxr-xr-x 1 Rajes 197609 1374 May 20 11:27 pre-push.sample*
-rwxr-xr-x 1 Rajes 197609 4898 May 20 11:27 pre-rebase.sample*
-rwxr-xr-x 1 Rajes 197609 544 May 20 11:27 pre-receive.sample*
-rwxr-xr-x 1 Rajes 197609 1492 May 20 11:27 prepare-commit-msg.sample*
-rwxr-xr-x 1 Rajes 197609 2783 May 20 11:27 push-to-checkout.sample*
-rwxr-xr-x 1 Rajes 197609 3650 May 20 11:27 update.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ chmod +x .git/hooks/pre-commit
chmod: cannot access '.git/hooks/pre-commit': No such file or directory

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*    pre-apppatch.sample*    pre-push.sample*        push-to-checkout.sample*
commit-msg.sample*        pre-commit*              pre-rebase.sample*      update.sample*
fsmonitor-watchman.sample* pre-commit.sample*       pre-receive.sample*
post-update.sample*        pre-merge-commit.sample* prepare-commit-msg.sample*

Rajes@charanreddy MINGW64 ~/Desktop/diamond/.git/hooks (GIT_DIR!)
$ cd ..

Rajes@charanreddy MINGW64 ~/Desktop/diamond (.git)
$ cd ..

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ ls
README.md  cherryfile  firsttext.txt

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ touch a

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ vi a

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ vi a

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  a

nothing added to commit but untracked files present (use "git add" to track)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git add .
warning: in the working copy of 'a', LF will be replaced by CRLF the next time Git touches it

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git commit -m "a file commit after pre-commit dir"
cat: ': No such file or directory
X ERROR: Invalid commit message format.
Message must start with 'JIRA-XXX' (e.g., JIRA-123 Fix login bug)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git commit -m "JIRA-XXX a file commit after pre-commit dir"
cat: ': No such file or directory
X ERROR: Invalid commit message format.
Message must start with 'JIRA-XXX' (e.g., JIRA-123 Fix login bug)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$ git commit -m "JIRA-123 a file commit after pre-commit dir"
cat: ': No such file or directory
X ERROR: Invalid commit message format.
Message must start with 'JIRA-XXX' (e.g., JIRA-123 Fix login bug)

Rajes@charanreddy MINGW64 ~/Desktop/diamond (main)
$
```

Squash Merge vs Rebase Merge

- Show the difference between squash merge and rebase merge with evidence

Here squash merge is used same as merge but all commits from another branch will be save as single commit and that should be done manually, but in git merge the commit is automatically executed and in normal merge all commits from other branch will be merged to current branch

Syntax: `git merge --squash branchname`

```
Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git merge --squash slave
Auto-merging cherryfile
CONFLICT (add/add): Merge conflict in cherryfile
Auto-merging firsttext.txt
CONFLICT (add/add): Merge conflict in firsttext.txt
Squash commit -- not updating HEAD
Automatic merge failed; fix conflicts and then commit the result.

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ vi cherryfile

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ vi firsttext.txt

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git merge --squash --continue
fatal: --continue expects no arguments

usage: git merge [<options>] [<commit>...]
```

```
$ git log
commit d8d91bf314040467f0028089c4f248ee56605f57 (HEAD -> main)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 16:17:02 2026 +0530

    JIRA-22 committed after squash merging
```

Rebase merging

Same as squash merge but commits will be in step by step not single

It will show all commits from the old branch to new branch
Shows every single commit in a manner

```
MINGW64/c/Users/Rajes/desktop/diamond
Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git status
On branch slave
Your branch and 'origin/slave' have diverged,
and have 4 and 4 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)

nothing to commit, working tree clean

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git rebase main
warning: skipped previously applied commit 1cadb5a
hint: use --reapply-cherry-picks to include skipped commits
hint: disable this message with "git config advice.skippedcherry Picks false"
Successfully rebased and updated refs/heads/slave.

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 5 commits.
(use "git push" to publish your local commits)

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git merge slave
Updating 1f9947..d5e775a
Fast-forward
 cherryfile      | 2 +
 firsttext.txt   | 6 +++
 q               | 115 +++++
 3 files changed, 123 insertions(+)
 create mode 100644 cherryfile
 create mode 100644 firsttext.txt
 create mode 100644 q

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git log
commit d5e775a8a567d9afea28b4166fa93107a6d6051a (HEAD -> main, slave)
Author: xbox <saicharanreddy5678@outlook.com>
```

```
MINGW64/c/Users/Rajes/desktop/diamond
Successfully rebased and updated refs/heads/slave.

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git log
commit efee4c21facd8d5a691c6db6b16bf0361158ae85 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 16:37:01 2026 +0530

    JIRA-22 last commit before rebase

commit 1cadb5a98ab804c5f86d1e70b70e46a880dc6c56
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 16:19:27 2026 +0530

    JIRA-22 committed for 23

commit f54a86a995f2cf9a4ff672b91a8f8bf94682f730
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 14:21:46 2026 +0530

    parent c87d94cfcb88c284871619c2bcd477832eb283a9
    author xbox <saicharanreddy5678@outlook.com> 1779267106 +0530
    committer xbox <saicharanreddy5678@outlook.com> 1779275379 +0530

    commit for revert exp

    Revert "commit for revert exp"

    This reverts commit f617e854ba865fffd43858d80b36236c625a1927.

    using for detached head using commit id

    JIRA-22 merge

    JIRA-122 commit for squash merge

    JIRA-12 committed after creating pre-msg

commit c87d94cfcb88c284871619c2bcd477832eb283a9 (tag: v1.1.1)
```

```
MINGW64/c/Users/Rajes/desktop/diamond
create mode 100644 cherryfile
create mode 100644 firsttext.txt
create mode 100644 q

Rajes@charanreddy MINGW64 ~/desktop/diamond (main)
$ git log
commit d5e775a8a567d9afea28b4166fa93107a6d6051a (HEAD -> main, slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 16:37:01 2026 +0530

    JIRA-22 last commit before rebase

commit 2495392e70071d84f5ce22cef9e6c6e67b0b5430
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 14:21:46 2026 +0530

    parent c87d94cfc88c284871619c2bcd477832eb283a9
    author xbox <saicharanreddy5678@outlook.com> 1779267106 +0530
    committer xbox <saicharanreddy5678@outlook.com> 1779275379 +0530

    commit for revert exp

    Revert "commit for revert exp"

    This reverts commit f617e854ba865fffd43858d80b36236c625a1927.

    using for detached head using commit id

    JIRA-22 merge

    JIRA-122 commit for squash merge

    JIRA-12 committed after creating pre-msg

commit ee9bc5fe1b74dc6841e7c774df438a2b396b4991
Author: xbox <saicharanreddy5678@outlook.com>
Date: Wed May 20 12:45:10 2026 +0530

    commit
```

Recover Lost Commit

- Commit something, reset hard, and then recover it using git reflog
- I have committed a newly edited file and used git reset --hard to delete the commit from logs
And used git reflog to see the last change using the last id from reflog i used git reset - - hard reflogid to get back the commit

```
MINGW64/c/Users/Rajes/desktop/diamond
0c8a43e HEAD@{35}: checkout: moving from main to slave
a1f9947 HEAD@{36}: checkout: moving from slave to main

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reflog head-1

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reflog -1
1f753f5 (HEAD -> slave) HEAD@{0}: reset: moving to HEAD

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ ls
21 22 23 45.txt README.md a cherryfile file2.txt firsttext.txt q

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ cat 23
new kjjadijbsdhcb

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reset --hard 1f753f5
HEAD is now at 1f753f5 JIRA-22 this commit will be deleted

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reflog -1
1f753f5 (HEAD -> slave) HEAD@{0}: reset: moving to 1f753f5

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git log -1
commit 1f753f54e4aafa0ec7d046ab43c57ed8d7dac010 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 17:05:37 2026 +0530

    JIRA-22 this commit will be deleted

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$
```

```
MINGW64/c/Users/Rajes/desktop/diamond
Message must start with 'JIRA-XXX' (e.g., JIRA-123 Fix login bug)

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git commit -m "JIRA-22 this commit will be deleted"
[slave 1f753f5] JIRA-22 this commit will be deleted
1 file changed, 1 insertion(+)

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git status
On branch slave
Your branch and 'origin/slave' have diverged,
and have 12 and 4 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)

nothing to commit, working tree clean

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git log -1
commit 1f753f54e4aafa0ec7d046ab43c57ed8d7dac010 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 17:05:37 2026 +0530

    JIRA-22 this commit will be deleted

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reset --hard
HEAD is now at 1f753f5 JIRA-22 this commit will be deleted

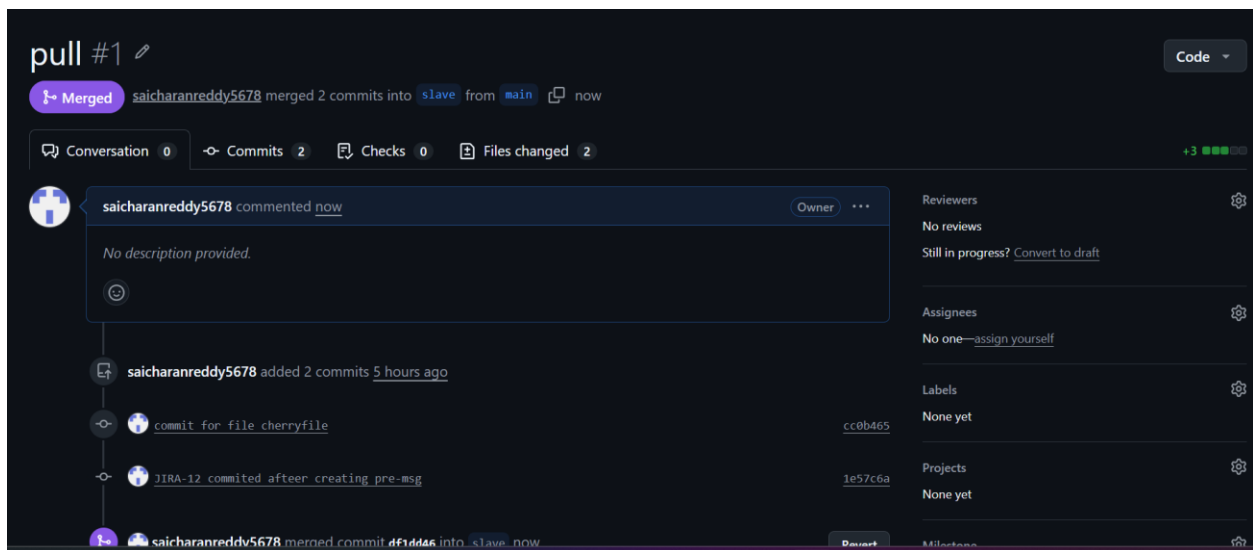
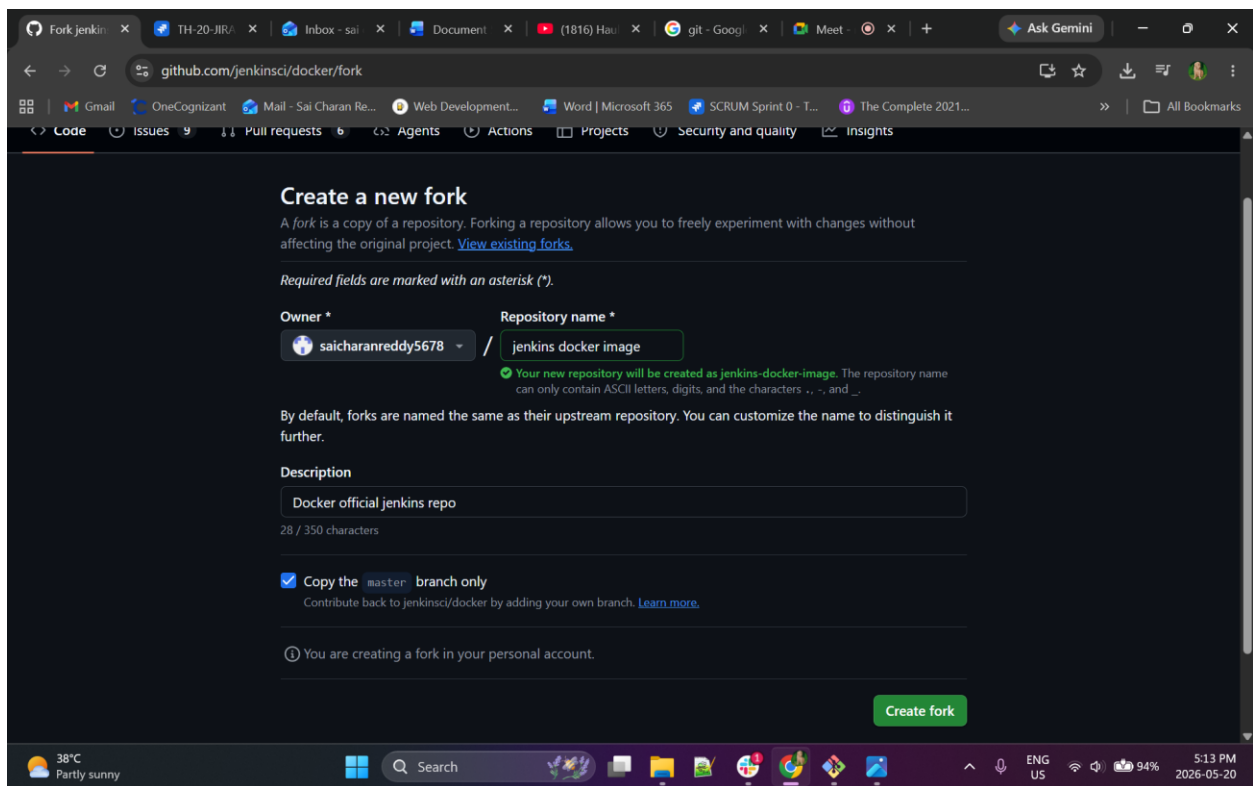
Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git log -1
commit 1f753f54e4aafa0ec7d046ab43c57ed8d7dac010 (HEAD -> slave)
Author: xbox <saicharanreddy5678@outlook.com>
Date:   Wed May 20 17:05:37 2026 +0530

    JIRA-22 this commit will be deleted

Rajes@charanreddy MINGW64 ~/desktop/diamond (slave)
$ git reflog
1f753f5 (HEAD -> slave) HEAD@{0}: reset: moving to HEAD
```

Fork & Pull Request Workflow

- Fork a repo, make a change, and submit a pull request to the original repo
Forked a repo



Cloning sub dir using sparse checkout

I have used syntax

Git clone --filter=blob:none --sparse git link

cd sparserepo

git sparse-checkout set r1/r3

((git sparse-checkout set --no-cone r1/r3 we can use this syntax for just the required subdir it excludes other directories))

```
mobaxterm backup:21p
Rajes@charanreddy MINGW64 ~/desktop
$ git clone --filter=blob:none --sparse git@github.com:saicharanreddy5678/sparserepo.git
Cloning into 'sparserepo'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 12 (delta 2), reused 8 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (12/12), done.
Resolving deltas: 100% (2/2), done.
remote: Enumerating objects: 2, done.
remote: Counting objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (2/2), 63 bytes | 63.00 KiB/s, done.
Updating files: 100% (2/2), done.
```

```
Rajes@charanreddy MINGW64 ~/desktop
$ cd sparserepo

Rajes@charanreddy MINGW64 ~/desktop/sparserepo (main|SPARSE)
$ git sparse-checkout set r1/r3
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (1/1), 41 bytes | 41.00 KiB/s, done.
Updating files: 100% (2/2), done.

Rajes@charanreddy MINGW64 ~/desktop/sparserepo (main|SPARSE)
$ ls
README.md  'no repo'  r1/
```